Hive

What is Data warehouse and Data Lake ?

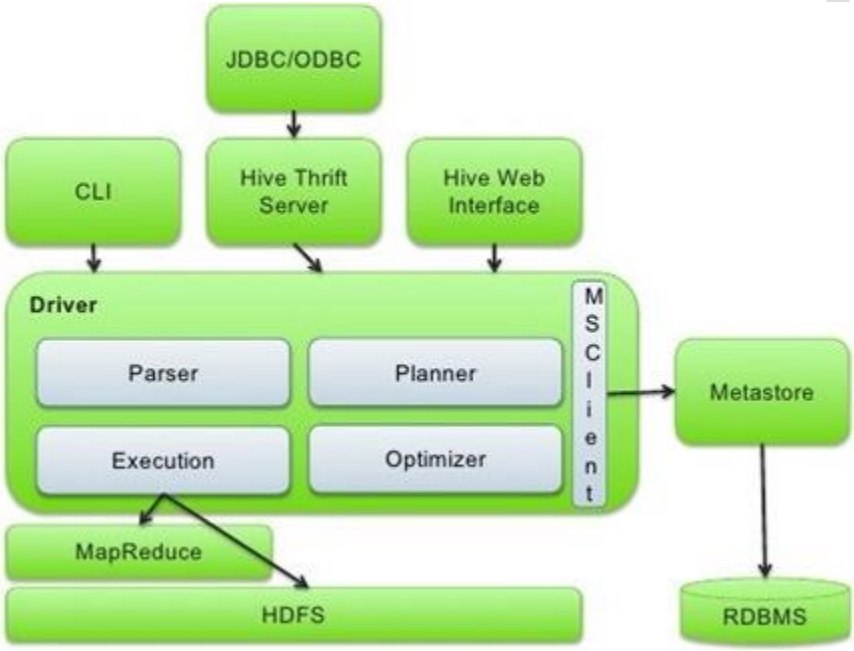
Data Warehouse	Data Lake
Comes data from multiple sources	Comes data from multiple sources
Structured data / Processed data before store in Data warehouse	Centralized storage of structured and unstructured data i.e Row Data
Data is processed so we can used Directly for Analysis	First we need to process and then analyze
Storing data is time consuming and cost	Expensive
Costly to make changes	We can change easily
Schema Define before store	Schema can define after store
Uses ETL	Uses ELT
High Data Quality	Low Data Quality
Purpose of data (in use)	Not yet defined
User – Business Professionals	User : Data Scientist
Ex. Hive	Ex. S3

What is Hive?

- ✓ Hive is the Data warehouse solution built top of hadoop.
- ✓ It is designed by facebook in 2007.

- ✓ Today Hive is under apache project.
- ✓ It provides SQL like Query named as Hive QL
- ✓ It is data warehouse tool
- ✓ HQL is not case sensitive
- ✓ Execution engine of Hive is by default MR.
- ✓ Ability to bring structure to various data.
- ✓ Access to files which is stored on HDFS or Hive.
- ✓ Even querying small amount of data may take minutes.

Hive Architecture



Component of Hive Architecture :

Metastore :

- ✓ To support features like schema and data partitioning Hive keeps its metadata in a relational database.
- ✓ It store metadata of table such as schema , (columns etc) and location along with partition metadata such as partition column n all.
- ✓ It stores data like MySQL etc.
- ✓ It help to driver for execution.
- ✓ Default data store in derby database which is embedded SQL DB.
- ✓ Schema is not shared by users. For each user having different schema.

Driver :

- ✓ It is like a controller which receives query or HQL statement and then driver start execute that by creating new session.
- ✓ It monitor progress of execution.

Compiler :

- ✓ It compile the query and convert that into execution plan. Plan contains task and steps which needed to perform by MR to get output.

Optimizer :

- ✓ It performs various transformation on execution plan to provided optimized DAG. Single join or all.

Executor :

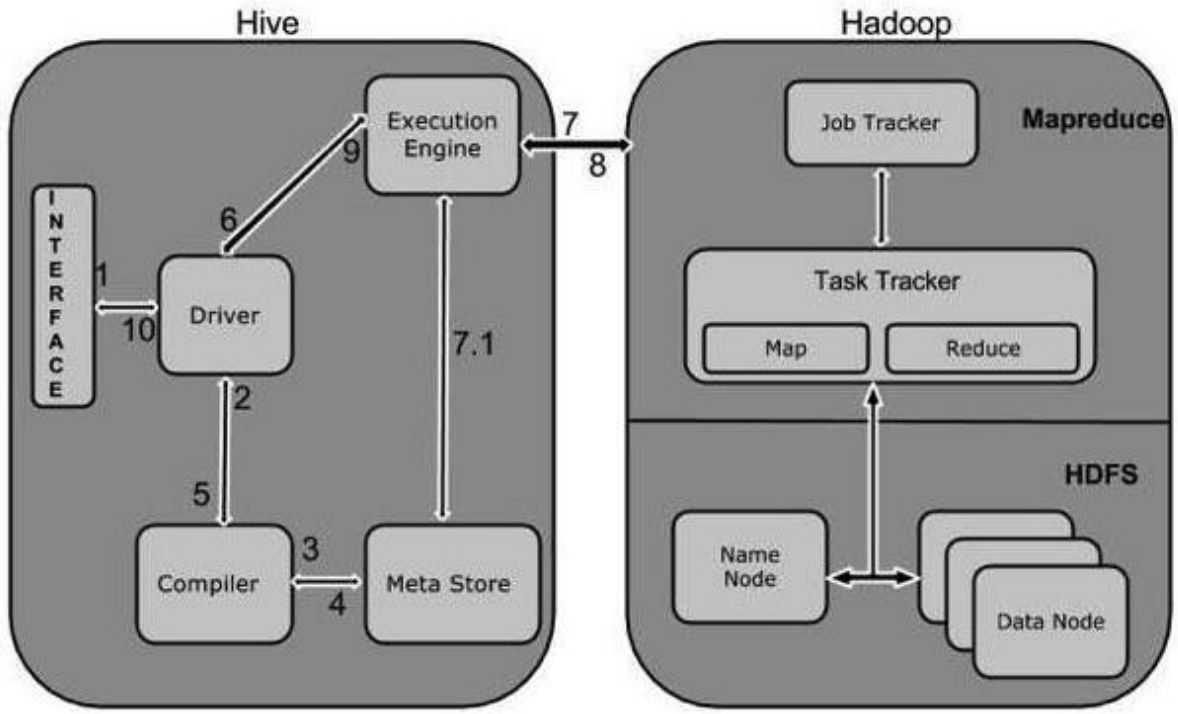
- ✓ Once compilation and optimization done then executor executes the task .

CLI : Command line interface provided user interface for external user to interact with hive.

Thift server :

It allow external client to ineract with hive over network similar to JDBC or ODBC

Query execution in Hive :



1. Hive interface such as CLI,UI or webUi send query to Driver to execute →
2. Driver takes help of compiler to parse query to check syntax and query plan →
3. Compiler send request to Metastore →
4. Metastore send the metadata as response to compiler →
5. Compliler check the requirement and send plan to driver →
6. Driver send execution plan to executor →
7. Execution of task internally is nothing but execution of MR job. Executin engine submit job to task tracker / resource manager and all execution happens →
8. Execution engine receives result from NM.
9. Execution engine send that result to driver.
10. Driver send that result to Hive Interface.

Hive Interactive Mode :

3 mode of interaction

1. Hive shell
2. Web UI
3. JDBC connection.

Interactive : hive shell -> run query

Non interactive : HQL → hive –f name.hql

Feature of Hive

i. Framework
Apache Hive is built on top of HDFS
ii. Large datasets
However, in distributed storage, it helps to query large datasets residing.
iii. Warehouse
Also, we can say Hive is a distributed data warehouse.
iv. Language
Queries data using a SQL-like language called HiveQL (HQL).
v. Declarative language
HiveQL is a declarative language like SQL.
vi. Table structure
Table structure/s is/are similar to tables in a relational database.
vii. Multi-user
Multiple users can simultaneously query the data using Hive-QL.
viii. Data Analysis
However, to perform more detailed data analysis, Hive allows writing custom MRframework processes.

Sayu Softtech

ix. ETL support

Also, it is possible to extract/transform/load (ETL) Data easily.

x. Data Formats

Moreover, Hive offers the structure on a variety of data formats.

xi. Storage

Hive allows access files stored in HDFS. Also, similar others data storage systems such as HBase.

x. Format conversion

Moreover, it allows converting the variety of format from to within Hive. Although, it is very simple and possible.

Limitations of Hive

i. OLTP Processing issues

However, Hive is not designed for Online transaction processing (OLTP). Although, we can use it for the Online Analytical Processing (OLAP).

ii. No Updates

It does not support updates and deletes, however, it does support overwriting or apprehending data.

iii. Subqueries

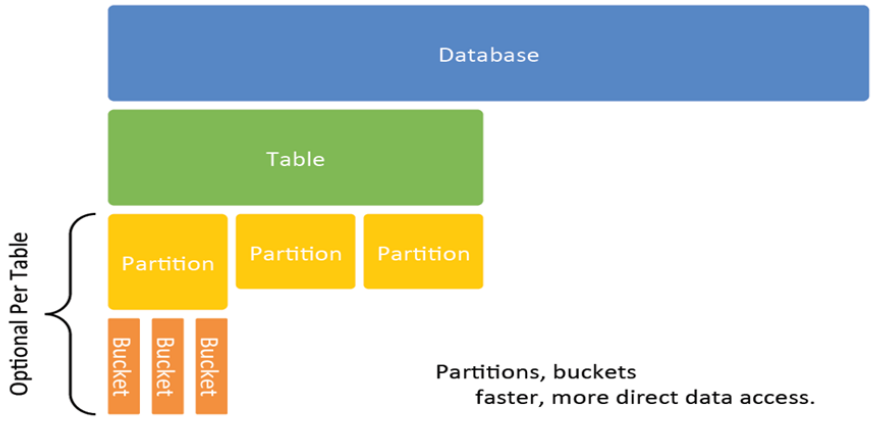
Basically, in Hive, Subqueries are not supported.

Hive Data Types :

SQL Datatypes
INT
TINYINT/SMALLINT/BIGINT
BOOLEAN
FLOAT
DOUBLE
STRING
BINARY
TIMESTAMP
ARRAY, MAP, STRUCT, UNION
DECIMAL
CHAR
VARCHAR
DATE

SQL Semantics
SELECT, LOAD, INSERT from query
Expressions in WHERE and HAVING
GROUP BY, ORDER BY, SORT BY
CLUSTER BY, DISTRIBUTE BY
Sub-queries in FROM clause
GROUP BY, ORDER BY
ROLLUP and CUBE
UNION
LEFT, RIGHT and FULL INNER/OUTER JOIN
CROSS JOIN, LEFT SEMI JOIN
Windowing functions (OVER, RANK, etc.)
Sub-queries for IN/NOT IN, HAVING
EXISTS / NOT EXISTS
INTERSECT, EXCEPT

Hive Abstraction :



Hive Basic Operations :

- ✓ DDL:
- ✓ Create database.
- ✓ Use database.
- ✓ Show databases.
- ✓ Create table.
- ✓ Etc.

Types of Tables in Hive :

There is 2 types of tables in Hive.

1.Internal or Hive managed table

2.External Tables.

Internal Table :

- ✓ It is default table in hive.
- ✓ It is internally managed by hive. It keeps all data under /user/hive/warehouse .
- ✓ When we drop the table data as well as metadata will drop.
- ✓ Support Truncate
- ✓ It support ACID property based on some conditions.

Example :

Sayu Softtech

CREATE TABLE weather (wban INT, date STRING, precip INT) ROW FORMAT DELIMITED FIELDS TERMINATED BY ‘,’;

LOAD DATA INPATH ‘hdfs:/data/2012.txt’ INTO TABLE weather;

External Table :

- ✓ We need to put external keyword while creating hive table.
- ✓ When we create the external hive table that time we need to give location path of hdfs to store the data.
- ✓ It is not managed by Hive.
- ✓ When we drop the external table only metadata will drop and the actual data will remain same.

Example .

CREATE EXTERNAL TABLE weatherext (wban INT, date STRING) ROW FORMAT DELIMITED FIELDS TERMINATED BY ‘,’ LOCATION ‘ /hive/data/weatherext’;

LOAD DATA INPATH ‘hdfs:/data/2012.txt’ INTO TABLE std;

Hive Querying :

- ✓ DDL.
- ✓ Data load.
- ✓ Hive View.

Hive Script (HQL) :

- ✓ Write the queries in one file and save with .hql externsion and run using

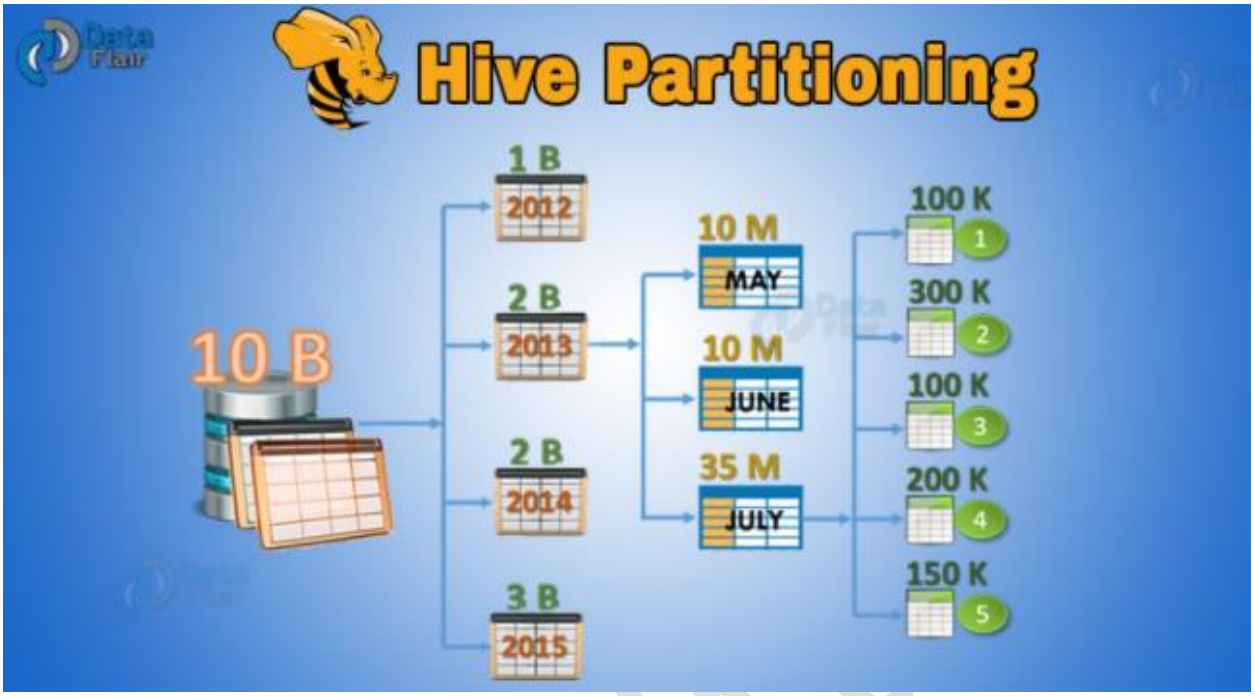
$hive –f name.hql

Change hive execution engine.

- ✓ Hive > set hive. execution. engine=spark;
- ✓ Hive > set hive. execution. engine=mr;

Hive Partition :

Sayu Softtech



- ✓ It is nothing but partition of table.
- ✓ In is performance improve technique.
- ✓ It decrease the I/O operations.
- ✓ It create physical dir same as data in city as city name.
- ✓ Normally we do partition on date.
- ✓ If multiple column in partition it will combine that records.

Exm.

create table bank (ac int,name string,city string)

partitioned by (country string)

row format delimited

fields terminated by ',';

data : file 1

111,abc,Hyd,India

222,bcd,Mum,India

333,cde,Pun,India

Sayu Soft

data : file 2

444,efg,Cal,USA

555,xyz,NY,USA

LOAD DATA LOCAL INPATH '/home/cloudera/Desktop/hive/india.csv' INTO TABLE bank partition (country='India');

LOAD DATA LOCAL INPATH '/home/cloudera/Desktop/hive/usa.csv' INTO TABLE bank partition (country='USA');

```
[cloudera@quickstart hive]$ hadoop fs -ls /user/hive/warehouse/test.db/bank
Found 1 items
drwxrwxrwx   - cloudera supergroup          0 2020-12-08 21:56 /user/hive/warehouse/test.db/bank/country=India
[cloudera@quickstart hive]$ hadoop fs -ls /user/hive/warehouse/test.db/bank/country=India
Found 3 items
-rwxrwxrwx   1 cloudera supergroup         54 2020-12-08 21:53 /user/hive/warehouse/test.db/bank/country=India/india.csv
-rwxrwxrwx   1 cloudera supergroup         54 2020-12-08 21:55 /user/hive/warehouse/test.db/bank/country=India/india_copy_1.csv
-rwxrwxrwx   1 cloudera supergroup         36 2020-12-08 21:56 /user/hive/warehouse/test.db/bank/country=India/india_copy_2.csv
You have new mail in /var/spool/mail/cloudera
[cloudera@quickstart hive]$
```

```
[cloudera@quickstart hive]$ hadoop fs -ls /user/hive/warehouse/test.db/bank/country=India
Found 3 items
-rwxrwxrwx   1 cloudera supergroup         54 2020-12-08 21:53 /user/hive/warehouse/test.db/bank/country=India/india.csv
-rwxrwxrwx   1 cloudera supergroup         54 2020-12-08 21:55 /user/hive/warehouse/test.db/bank/country=India/india_copy_1.csv
-rwxrwxrwx   1 cloudera supergroup         36 2020-12-08 21:56 /user/hive/warehouse/test.db/bank/country=India/india_copy_2.csv
You have new mail in /var/spool/mail/cloudera
[cloudera@quickstart hive]$ hadoop fs -cat /user/hive/warehouse/test.db/bank/country=India/india.csv
111,abc,India,Hyd
222,bcd,India,Mum
333,cde,India,Pun
You have new mail in /var/spool/mail/cloudera
[cloudera@quickstart hive]$ hadoop fs -cat /user/hive/warehouse/test.db/bank/country=India/india_copy_1.csv
111,abc,Hyd,India
222,bcd,Mum,India
333,cde,Pun,India
[cloudera@quickstart hive]$ hadoop fs -cat /user/hive/warehouse/test.db/bank/country=India/india_copy_2.csv
111,abc,Hyd
222,bcd,Mum
333,cde,Pun
[cloudera@quickstart hive]$
```

#partition on multiple columns

create table bank (ac int,name string)

partitioned by (country string,city string)

row format delimited

fields terminated by ',';

111,abc,India,Hyd

222,bcd,India,Mum

333,cde,India,Pun

444,abc,India,Hyd

555,bcd,India,Mum

666,cde,India,Pun

LOAD DATA LOCAL INPATH '/home/cloudera/Desktop/hive/india.csv' INTO TABLE bank partition (country='India',city='Pun');

LOAD DATA LOCAL INPATH '/home/cloudera/Desktop/hive/india.csv' INTO TABLE bank partition (country='India',city='Mum');

LOAD DATA LOCAL INPATH '/home/cloudera/Desktop/hive/india.csv' INTO TABLE bank partition (country='India',city='Hyd');

```
[cloudera@quickstart hive]$ hadoop fs -ls /user/hive/warehouse/test.db/bank/country=India
Found 3 items
drwxrwxrwx   - cloudera supergroup          0 2020-12-08 22:41 /user/hive/warehouse/test.db/bank/country=India/city=Hyd
drwxrwxrwx   - cloudera supergroup          0 2020-12-08 22:41 /user/hive/warehouse/test.db/bank/country=India/city=Mum
drwxrwxrwx   - cloudera supergroup          0 2020-12-08 22:40 /user/hive/warehouse/test.db/bank/country=India/city=Pun
You have new mail in /var/spool/mail/cloudera
[cloudera@quickstart hive]$ hadoop fs -ls /user/hive/warehouse/test.db/bank/country=India/city=Pun
Found 1 items
-rwxrwxrwx   1 cloudera supergroup        108 2020-12-08 22:40 /user/hive/warehouse/test.db/bank/country=India/city=Pun/india.csv
[cloudera@quickstart hive]$ hadoop fs -cat /user/hive/warehouse/test.db/bank/country=India/city=Pun/india.csv
111,abc,India,Hyd
222,bcd,India,Mum
333,cde,India,Pun
444,abc,India,Hyd
555,bcd,India,Mum
666,cde,India,Pun
[cloudera@quickstart hive]$
```

Types of Hive Partitioning

There are two types of Partitioning in Apache Hive-

- Static Partitioning
- Dynamic Partitioning

1.Static partition :

- ✓ Insert input data files individually into a partition table is Static Partition.
- ✓ Usually when loading files (big files) into **Hive tables** static partitions are preferred.
- ✓ Static Partition saves your time in loading data compared to dynamic partition.
- ✓ You “statically” add a partition in the table and move the file into the partition of the table.
- ✓ We can alter the partition in the static partition.
- ✓ You can perform Static partition on Hive Manage table or external table.
- ✓ BY default static partition is strict or enable.
- ✓ To use dynamic partition we need to make it non-strict.

2.Dymanic Partition

- ✓ Single insert to partition table is known as a dynamic partition.
- ✓ Usually, dynamic partition loads the data from the non-partitioned table.
- ✓ Dynamic Partition takes more time in loading data compared to static partition.
- ✓ When you have large data stored in a table then the Dynamic partition is suitable.
- ✓ If you want to partition a number of columns but you don't know how many columns then also dynamic partition is suitable.
- ✓ You can perform dynamic partition on hive external table and managed table.
- ✓ If you want to use the Dynamic partition in the hive then the mode is in non-strict mode

Disadvantages :

- ✓ If lots of small file takes time for group by operations.

SET hive.exec.dynamic.partition = true;
SET hive.exec.dynamic.partition.mode = nonstrict;

create non partition table:

create table bank (ac int,name string,country string,city string)
row format delimited
fields terminated by ',';

LOAD DATA into non partitioned table :
LOAD DATA LOCAL INPATH '/home/cloudera/Desktop/hive/india.csv'
INTO TABLE bank;

create partitioned table :

create table bank_Dy (ac int,name string)
partitioned by (country string,city string)
row format delimited
fields terminated by ',';

insert into bank_Dy partition(country,city) as select * from bank;

insert overwrite table bank_Dy partition(country, city) select ac, name,country,city from bank;

```
^C[cloudera@quickstart hive]$ hadoop fs -ls /user/hive/warehouse/test.db/bank_dy/country=India
Found 3 items
drwxrwxrwx   - cloudera supergroup          0 2020-12-08 23:11 /user/hive/warehouse/test.db/bank_dy/country=India/city=Hyd
drwxr-xr-x   - cloudera supergroup          0 2020-12-08 23:11 /user/hive/warehouse/test.db/bank_dy/country=India/city=Mum
drwxrwxrwx   - cloudera supergroup          0 2020-12-08 23:11 /user/hive/warehouse/test.db/bank_dy/country=India/city=Pun
[cloudera@quickstart hive]$ hadoop fs -ls 2020-12-08 23:11 /user/hive/warehouse/test.db/bank_dy/country=India/city=Hyd
```

```
[cloudera@quickstart hive]$ hadoop fs -ls /user/hive/warehouse/test.db/bank_dy/country=India/city=Hyd
Found 1 items
-rwxrwxrwx   1 cloudera supergroup         16 2020-12-08 23:11 /user/hive/warehouse/test.db/bank_dy/country=India/city=Hyd/000000_0
You have new mail in /var/spool/mail/cloudera
[cloudera@quickstart hive]$ hadoop fs -cat  /user/hive/warehouse/test.db/bank_dy/country=India/city=Hyd/000000_0
111,abc
444,abc
[cloudera@quickstart hive]$
```

Bucketing :

- ✓ Subdivision of partition is nothing but bucketing.
- ✓ We use partitioning when
  - When there is the limited number of partitions.
  - Or, while partitions are of comparatively equal size.
- ✓ But if number of partitions are more or data is not in equal size that time we use bucketing.

Features of Bucketing in Hive

Basically, this concept is based on hashing function on the bucketed column. Along with mod (by the total number of buckets).

- ✓ Where the hash_function depends on the type of the bucketing column.
- ✓ However, the Records with the same bucketed column will always be stored in the same bucket.
- ✓ Moreover, to divide the table into buckets we use CLUSTERED BY clause.
- ✓ Generally, in the table directory, each bucket is just a file, and Bucket numbering is 1-based.
- ✓ Along with Partitioning on Hive tables bucketing can be done and even without partitioning.
- ✓ Moreover, Bucketed tables will create almost equally distributed data file parts.
- ✓ Map-side joins will be faster on bucketed tables than non-bucketed tables, as the data files are equal sized **parts.**

Create bucketed table :

```
create table bank_b (ac int,name string,country string,city string)
clustered by (country) into 2 buckets
row format delimited
fields terminated by ',';
```

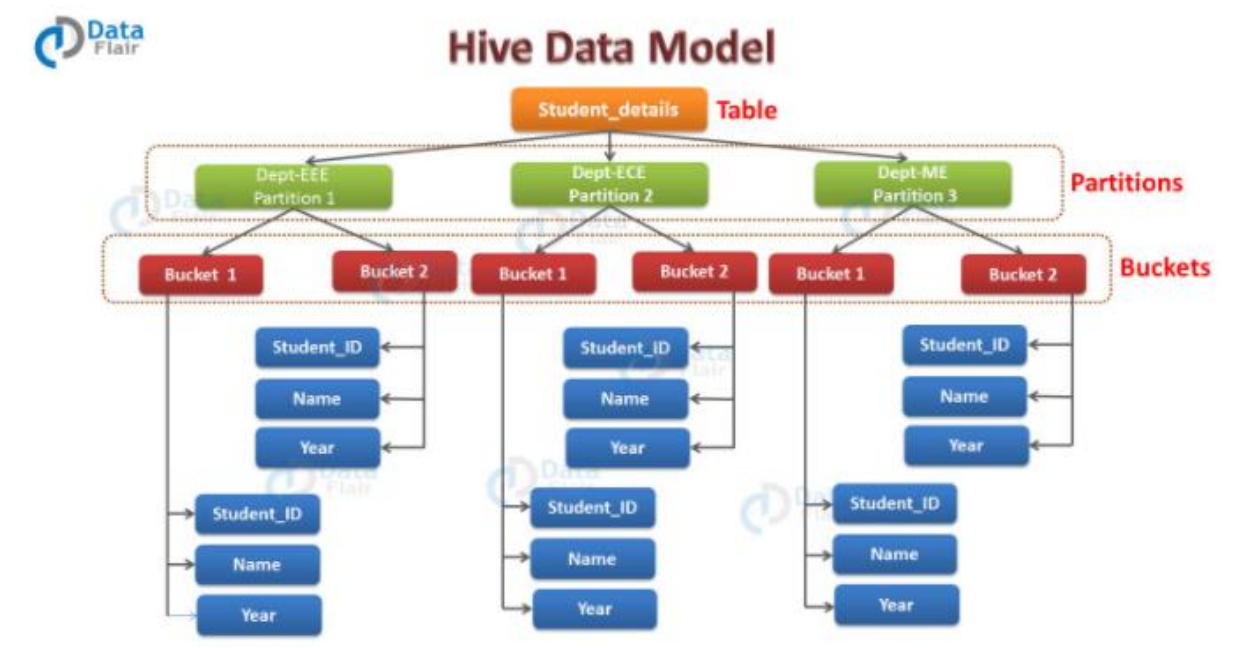
Note : At the time of clustered by you need to add column details in column list.

we can not load data into bucket table directly using LOAD command.
we need to set below property true.
set hive.enforce.bucketing = true

```
insert overwrite table bank_b select ac,name,country,city from bank;
```

```
[cloudera@quickstart hive]$ hadoop fs -ls /user/hive/warehouse/test.db/bank_b/
Found 2 items
-rwxrwxrwx   1 cloudera supergroup          0 2020-12-09 01:08 /user/hive/warehouse/test.db/bank_b/000000_0
-rwxrwxrwx   1 cloudera supergroup        108 2020-12-09 01:08 /user/hive/warehouse/test.db/bank_b/000001_0
You have new mail in /var/spool/mail/cloudera
```

Partition and Bucketing :



Hive Data Model

create table bank_pb (ac int,name string,city string)
partitioned by (country string)
clustered by (city) into 2 buckets
row format delimited
fields terminated by ',';
insert overwrite table bank_pb partition(country) select ac,name ,city, country from bank;

```
[cloudera@quickstart hive]$ hadoop fs -ls /user/hive/warehouse/test.db/bank_pb
Found 3 items
drwxrwxrwx   - cloudera supergroup          0 2020-12-09 01:19 /user/hive/warehouse/test.db/bank_pb/country=Hyd
drwxrwxrwx   - cloudera supergroup          0 2020-12-09 01:19 /user/hive/warehouse/test.db/bank_pb/country=Mum
drwxrwxrwx   - cloudera supergroup          0 2020-12-09 01:19 /user/hive/warehouse/test.db/bank_pb/country=Pun
[cloudera@quickstart hive]$ hadoop fs -ls /user/hive/warehouse/test.db/bank_pb/country=Mun
^CYou have new mail in /var/spool/mail/cloudera
[cloudera@quickstart hive]$ hadoop fs -ls /user/hive/warehouse/test.db/bank_pb/country=Mum
Found 2 items
-rwxrwxrwx   1 cloudera supergroup          0 2020-12-09 01:19 /user/hive/warehouse/test.db/bank_pb/country=Mum/000000_0
-rwxrwxrwx   1 cloudera supergroup         28 2020-12-09 01:19 /user/hive/warehouse/test.db/bank_pb/country=Mum/000001_0
[cloudera@quickstart hive]$ hadoop fs -cat /user/hive/warehouse/test.db/bank_pb/country=Mum/000001_0
555,bcd,India
222,bcd,India
You have new mail in /var/spool/mail/cloudera
[cloudera@quickstart hive]$
```

Some Advanced Concept :

Sort Merge Bucket :

Criteria :

1.each table in join must be bucketised.

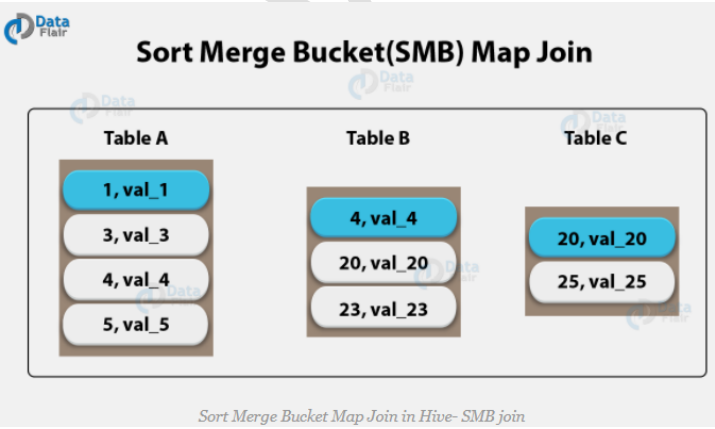
2.no. of buckets of big table must be divisible by number of buckets of remaining join tables or same.

3.bucket column and join column in the query must be same.

4. Need to set below properties.

```
> SET hive.input.format=>
org.apache.hadoop.hive.ql.io.BucketizedHiveInputFormat;
> SET hive.auto.convert.sortmerge.join=true;
> SET hive.optimize.bucketmapjoin=true;
> SET hive.optimize.bucketmapjoin.sortedmerge=true;
> SET hive.auto.convert.sortmerge.join.noconditionaltask=true;
```

- ✓ In hive , each mapper reads data from 1st bucket of first table and corresponding bucket from second table, in SMB join.
- ✓ We use this when both tables is large.
- ✓ We need to make sure all tables having same number of buckets.
- ✓ In the same way how the SQL joins Tables need to be bucketed. Hence, for other types of SQL, it cannot be used.
- ✓ Also, it is possible that Partition tables might slow down here.
- ✓ Mapper present and reducer 0.



Sort Merge Bucket Map Join in Hive- SMB join

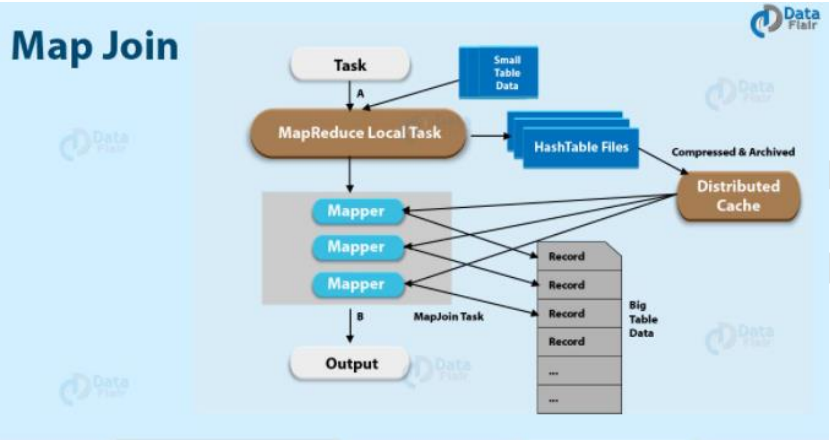
When we use SMB join?

- ✓ While all tables are Large.

- ✓ Also, while all tables are bucketed using the join columns.
- ✓ While by using the join columns, Sorted.
- ✓ Also, when the number of buckets is same as the number of all tables.

Map Join / MapSide Join / broadcast join :

- ✓ there is a major issue with that it there is too much activity spending on shuffling data around. So, as a result, that slows the Hive Queries. Hence, to speed up the Hive queries, we can use Map Join in Hive.
- ✓ we use Hive Map Side Join since one of the tables in the join is a small table and can be loaded into memory. So that a join could be performed within a mapper without using a Map/Reduce step.



Working :

Basically, before the original MapReduce task, its first step is to create a MapReduce local task. However, from HDFS this map/reduce task read data of the small table. Further save it into an in-memory hash table, then into a hash table file. Afterward, it moves the hash table file to the Hadoop Distributed Cache while original join MapReduce task starts, which will populate the file to each mapper's local disk. Hence, in this way, all the mapper can load this hash table file into the memory and then do the join in Map stage.
So, let's understand this with an example. let's suppose, for a join with big table A and small table B, for every mapper for table A, Table B is read completely. Since the smaller table is loaded into memory at first. Afterward, join is performed in the map phase of the MapReduce job, no reducer is needed and reduce phase is skipped. However, map joins in Hive are way faster than the regular joins since no reducers are necessary.

Hive mapjoin hint :

Select **/*+ MAPJOIN(b) */** a.key, a.value from a join b on a.key = b.key

to force to use the map join we can use "MAPJOIN".
At first, make sure below parameter is set to false(Default is true in Hive 0.13).

set hive.ignore.mapjoin.hint=false;
Then:
select /*+ MAPJOIN(a) */ a.* from passwords a, passwords2 b where a.col0=b.col0 ;

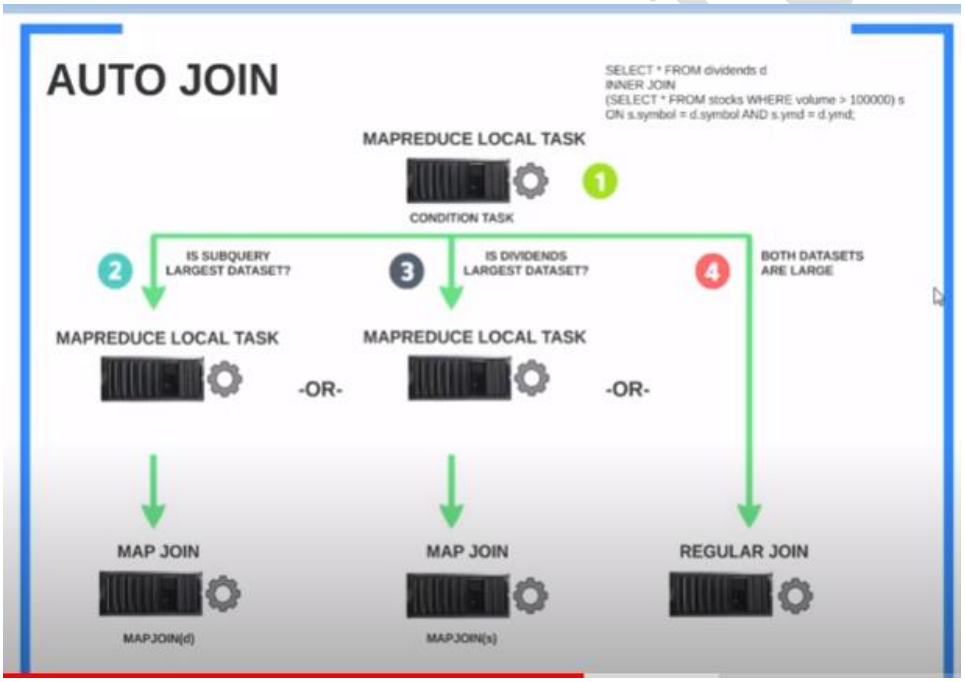
Auto Map Join :

If we don't give the mapjoin hint its nothing but auto map join.

For this we need to give below property then hive will take whole responsibility to perform map join .

```
hive> set hive.auto.convert.join=true;
hive> set hive.auto.convert.join.noconditionaltask=true;
hive> set hive.auto.convert.join.noconditionaltask.size=20971520
hive> set hive.auto.convert.join.use.nonstaged=true;
hive> set hive.mapjoin.smalltable.filesize = 30000000;
```

- At first, by default, it is starting from Hive 0.11, hive.auto.convert.join=true.



Hive Optimization :

1.Change execution engine

Tez-Execution Engine in Hive

2. Usage of Suitable File Format in Hive

ORCFILE File Formate – Hive Optimization Techniques, if we use appropriate file format on the basis of data. It will drastically increase our query performance. Basically, for increasing your query performance ORC file format is best suitable. Here, ORC refers to Optimized Row Columnar. That implies we can store data in an optimized way than the other file formats.

To be more specific, ORC reduces the size of the original data up to 75%. Hence, data processing speed also increases. On comparing to Text, Sequence and RC file formats, ORC shows better performance. Basically, it contains rows data in groups. Such as Stripes along with a file footer. Therefore, we can say when Hive is processing the data ORC format improves the performance.

3. Hive Partitioning
4. Bucketing in Hive
5. Cost-Based Optimization in Hive (CBO)

Cost-Based Optimization in Hive – Hive Optimization Techniques, before submitting for final execution Hive optimizes each Query's logical and physical execution plan. Although, until now these optimizations are not based on the cost of the query. However, CBO, performs, further optimizations based on query cost in a recent addition to Hive. That results in potentially different decisions: how to order joins, which type of join to perform, the degree of parallelism and others.

To use CBO, set the following parameters at the beginning of your query:

```
set hive.cbo.enable=true;
set hive.compute.query.using.stats=true;
set hive.stats.fetch.column.stats=true;
set hive.stats.fetch.partition.stats=true;
```

Then, prepare the data for CBO by running Hive's "analyze" command to collect various statistics on the tables for which we want to use CBO.

6. Hive Indexing

- ✓ Hive Index – Hive Optimization Techniques, one of the best ways is Indexing. To increase your query performance indexing will definitely help. Basically, for the original table use of indexing will create a separate called index table which acts as a reference.
- ✓ As we know, there are many numbers of rows and columns, in a **Hive table**. Basically, it will take a large amount of time if we want to perform queries only on some columns without indexing. Because queries will be executed on all the columns present in the table.
- ✓ Moreover, there is no need for the query to scan all the rows in the table while we perform a query on a table that has an index, it turned out as the major advantage of using indexing. Further, it checks the index first and then goes to the particular column and performs the operation.

Hive index :

When to use Indexing?

Indexing can be use under the following circumstances:

- If the dataset is very large.
- If the query execution is more amount of time than you expected.
- If a speedy query execution is required.
- When building a data model.

Types of Indexes in Hive

- Compact Indexing
- Bitmap Indexing

Bit map indexing was introduced in Hive 0.8 and is commonly used for columns with distinct values.

Compact indexing stores the pair of indexed column's value and its blockid.

Bitmap indexing stores the combination of indexed column value and list of rows as a bitmap. Building more number of indexes also degrade the performance of your query.

The following are the different operations that can be performed on Hive indexes:

- Creating index
- Showing index
- Alter index
- Dropping index

create index

```
create index i1 on table bank(ac) as
'org.apache.hadoop.hive.ql.index.compact.CompactIndexHandler' WITH DEFERRED REBUILD;

create index i2 on table bank(ac) as 'BITMAP' WITH DEFERRED REBUILD;

show index on bank;

show formatted index on bank;

DROP INDEX IF EXISTS i1 ON bank;
```

Hive File Format :

Hive supports several file formats:

- Text File
- SequenceFile
- RCFile
- Avro Files
- ORC Files
- Parquet
- Custom INPUTFORMAT and OUTPUTFORMAT

stored as TEXTFILE.

stored as SEQUENCEFILE.

stored as RCFILE

stored as ORC

avro :

```
CREATE TABLE kst
 PARTITIONED BY (ds string)
 ROW FORMAT SERDE
 'org.apache.hadoop.hive.serde2.avro.AvroSerDe'
 STORED AS INPUTFORMAT
 'org.apache.hadoop.hive.ql.io.avro.AvroContainerInputFormat'
 OUTPUTFORMAT
 'org.apache.hadoop.hive.ql.io.avro.AvroContainerOutputFormat'
 TBLPROPERTIES (
   'avro.schema.url'='http://schema_provider/kst.avsc');
```

Parquet :

```
CREATE TABLE parquet_test (
 id int,
 str string,
 mp MAP<STRING,STRING>,
 lst ARRAY<STRING>,
 strct STRUCT<A:STRING,B:STRING>)
PARTITIONED BY (part string)
ROW FORMAT SERDE 'parquet.hive.serde.ParquetHiveSerDe'
 STORED AS
 INPUTFORMAT 'parquet.hive.DeprecatedParquetInputFormat'
```

Sayu Softtech

```
OUTPUTFORMAT 'parquet.hive.DeprecatedParquetOutputFormat';
```

OR

Hive 0.13 and later

```
CREATE TABLE parquet_test (
 id int,
 str string,
 mp MAP<STRING,STRING>,
 lst ARRAY<STRING>,
 strct STRUCT<A:STRING,B:STRING>)
PARTITIONED BY (part string)
STORED AS PARQUET;
```

Hive complex data type :

Complex Data Types

- **arrays**: ARRAY<data_type>
- **maps**: MAP<primitive_type, data_type>
- **structs**: STRUCT<col_name : data_type [COMMENT col_comment], ...>

#1 Array

```
create table std(roll int,mark array <int>)
row format delimited
fields terminated by ','
collection items terminated by '|';
```

10,50|60|70|80|90

load data:

```
load data LOCAL INPATH '/home/cloudera/Desktop/hive/ar.csv' into table std;
```

array <string>

collection items terminated by '$'

Access : pf[0]

```
select roll,mark[1] from std;
```

Sayu Soft Tech

```
hive> load data LOCAL INPATH '/home/cloudera/Desktop/hive/ar.csv' into table std;
Loading data to table test.std
Table test.std stats: [numFiles=1, totalSize=18]
OK
Time taken: 0.668 seconds
hive> select * from std;
OK
10      [50,60,70,80,90]
Time taken: 0.083 seconds, Fetched: 1 row(s)
hive> select roll,mark[1] from std;
OK
10      60
Time taken: 0.118 seconds, Fetched: 1 row(s)
```

Map :
var map<string,int>
collection items terminated by '$'
map keys terminated by '#'

pf["pf"

example :

```
create table std1(roll int,mark map <string,int>)
row format delimited
fields terminated by ','
collection items terminated by '|'
map keys terminated by '#';
```

data : 10,marathi#20|hindi#50

load data:
load data LOCAL INPATH '/home/cloudera/Desktop/hive/mp.csv' into table std1;

check data :
select roll, mark["hindi"] from std1;

Sayu Softtech

```
hive> create table std1(roll int,mark map <string,int>)
    > row format delimited
    > fields terminated by ','
    > collection items terminated by '|'
    > map keys terminated by '#';
OK
Time taken: 0.172 seconds
hive> load data LOCAL INPATH '/home/cloudera/Desktop/hive/mp.csv' into table std1;
Loading data to table test.std1
Table test.std1 stats: [numFiles=1, totalSize=23]
OK
Time taken: 0.491 seconds
hive> select * from std1;
OK
10      {"marathi":20,"hindi":50}
Time taken: 0.078 seconds, Fetched: 1 row(s)
hive> select roll, mark["hindi"] from std1;
OK
10      50
Time taken: 0.111 seconds, Fetched: 1 row(s)
hive>
```

Struct :

add struct<city:string,state:string,pin:biint>
collection items terminated by '$'
map keys terminated by '#'

Access : select address.city
example :
create table std2(roll int,address struct<city:string,state:string,pin:int>)
row format delimited
fields terminated by ','
collection items terminated by '|';

data : 10,Pune|Maharashtra|411014

load data:
load data LOCAL INPATH '/home/cloudera/Desktop/hive/st.csv' into table std2;

Sayu Soft

```
hive> create table std2(roll int,address struct<city:string,state:string,pin:int>)
    > row format delimited
    > fields terminated by ','
    > collection items terminated by '|';
OK
Time taken: 0.095 seconds
hive> load data LOCAL INPATH '/home/cloudera/Desktop/hive/st.csv' into table std2;
Loading data to table test.std2
Table test.std2 stats: [numFiles=1, totalSize=27]
OK
Time taken: 0.454 seconds
hive> select * from std2;
OK
10      {"city":"Pune","state":"Maharashtra","pin":411014}
Time taken: 0.099 seconds, Fetched: 1 row(s)
hive> select roll,address.pin from std2;
OK
10      411014
Time taken: 0.078 seconds, Fetched: 1 row(s)
hive>
```

Some more things :

Hive Analytical Functions :

Hadoop Hive analytic functions compute an aggregate value that is based on a group of rows. A Hadoop Hive HQL analytic function works on the group of rows and ignores the NULL in the data if you specify.

1. RANK
2. DENSE_RANK
3. ROW_NUMBER

```
create table emp (id int,mane string,des string,dept string,sal double)
row format delimited
fields terminated by ',';
```

```
#id,name,design,dept,sal
1111,aba,dir,sal,30000
2222,abb,mmg,sal,24000
3333,abc,amg,sal,20000
4444,abd,acc,aco,15000
5555,abe,aco,aco,15000
6666,abf,amg,aco,25000
7777,abg,mgr,sal,24000
8888,abh,aco,aco,27000
9999,abi,sco,aco,28000
```

LOAD DATA LOCAL INPATH '/home/cloudera/Desktop/hive/sal' INTO TABLE EMP;
#1. ROW_NUMBER()

select id,mane,dept,sal,row_number() OVER (PARTITION BY dept order by sal desc) as rm from emp;

9999	abi	aco	28000.0	1
8888	abh	aco	27000.0	2
6666	abf	aco	25000.0	3
5555	abe	aco	15000.0	4
4444	abd	aco	15000.0	5
1111	aba	sal	30000.0	1
7777	abg	sal	24000.0	2
2222	abb	sal	24000.0	3
3333	abc	sal	20000.0	4

#2. RANK()
select id,mane,dept,sal,RANK() OVER (PARTITION BY dept order by sal desc) as rm from emp;

9999	abi	aco	28000.0	1
8888	abh	aco	27000.0	2
6666	abf	aco	25000.0	3
5555	abe	aco	15000.0	4
4444	abd	aco	15000.0	4
1111	aba	sal	30000.0	1
7777	abg	sal	24000.0	2
2222	abb	sal	24000.0	2
3333	abc	sal	20000.0	4

#3. DENS_RANK()
select id,mane,dept,sal,DENS_RANK() OVER (PARTITION BY dept order by sal desc) as rm from emp;

explode and lateral view :
explode
id
[1,2,3]
[1,2]

result:
1
2
3
1
2
select explode(id) from tab_name.
auther book

ram [a,b,c]

select auther,book from table_name lateral view explode(book) dummy as dummy_books_name.
ram a
ram b
ram c
==========

Sayu Softtech